Files

File handling in Python refers to the process of working with files, including reading from and writing to files. Python provides built-in functions and modules to handle file operations efficiently.

Types of Files

1) **Text Files**: These files contain plain text data. You can read and write text files using the built-in file handling functions. Text files have extensions like .txt, .csv, and .log.
2) **Binary Files**: Binary files store data in a binary format, such as images, audio, video, or serialized objects. They cannot be directly read as plain text. Examples include image files (.jpg, .png), audio files (.mp3, .wav), and video files (.mp4, .avi).

File Paths

In Python, a file path is a string that specifies the location of a file on the file system. It allows you to identify the directory structure and the file name to access or manipulate files.

The following fundamental rules enable applications to create and process valid names for files and directories in both Windows and Linux operating systems unless explicitly specified:

- Use a period to separate the base file name from the extension in the file name.
- In Windows use backslash (\) and in Linux use forward slash (/) to separate the components of a path. Eg: D:\Python programs\gui1.py
- File and Directory names in Windows are not case sensitive while in Linux it is case sensitive.
- In Windows, drive letters are not case-sensitive. For example, "D:\" and "d:\" refer to the same drive.
- The reserved characters that should not be used in naming files and directories are <,>,:, ", /, \, |, ? and *.
- In Windows Operating system reserved words like CON, PRN, AUX, NUL, COM1, COM2, LPT1, LPT2, etc should not be used to name files and directories.

Fully Qualified path and relative path

Absolute Paths or Fully qualified path: An absolute file path provides the complete location of a file or directory from the root of the file system.It starts from the root directory, such as 'C:\' on Windows or '/' on Unix-based systems.

Example absolute paths:

Windows: 'C:\Users\Username\Documents\myfile.txt'
Unix-based: '/home/username/documents/myfile.txt'

Relative Paths: A relative file path is defined in relation to the current working directory. It provides a path to a file or directory relative to your current location in the file system.

In relative file paths in Python, the single dot (.) and double dot (..) have specific meanings and are used to refer to directories relative to the current directory.

1. Single Dot (.):In a relative file path, a single dot (.) represents the current directory. It is used to indicate that the file or directory is located in the same directory as the script or working directory.

Example: Assuming the script is located in the directory `C:¥Users¥Username¥Documents`, then `'.¥myfile.txt'` represents`C:¥Users¥Username¥Documents¥myfile.txt`

2. Double Dot (..):In a relative file path, two dots (..) represent the parent directory. It is used to indicate that the file or directory is located in the parent directory of the current directory.

Example: Assuming the script is located in the directory `C:¥Users¥Username¥Documents`, then `'..¥myfile.txt'` represents`C:¥Users¥Username¥myfile.txt`

Creating and Reading Text data

Python provides built-in functions for opening a file, reading from a file, writing to a file, and closing a file.

Creating and opening text files

To work with a file, you need to open it first using the open() function. It takes two parameters: the file with path and the mode. The mode specifies the purpose of opening the file, such as reading, writing, or appending.

The syntax of open() function is given below.

file_handler = open(filename, mode)

1. **Read Mode ('r'):** This is the default mode when opening a file. It allows you to read the contents of the file.

   Eg :file = open('filename.txt', 'r')

2. **Write Mode ('w')**: This mode allows you to write data to a file. If the file exists, it truncates the existing content before writing. If the file doesn't exist, it creates a new file.

   Eg: file = open('filename.txt', 'w')

3. **Append Mode ('a'):** This mode allows you to append new data to the end of an existing file. If the file doesn't exist, it creates a new file.

Eg: file = open('filename.txt', 'a')

4. **Exclusive Creation Mode ('x'):** This mode is used to create a new file, but it fails if the file already exists. It raises a FileExistsError if the file exists.

   Eg: file = open('filename.txt', 'x')

5. **r+** : It Opens the file for both reading and writing

   Eg: file = open('filename.txt', 'r+')

6. **w+:** Opens the file for reading and writing.If the file does not exist it creates a new file. If a file already exists then it will get overwritten.

   Eg: file = open('filename.txt', 'w+')

7. **a+ :**Opens the file for reading and appending.If a file already exists, the data is appended. If the file does not exist it creates a new file.

   Eg: file = open('filename.txt', 'a+')

8. **rb:**Opens the binary file in read-only mode.

   Eg: file = open('filename.txt', 'rb')

9. **wb :**Opens the file for writing the data in binary format.

   Eg: file = open('filename.txt', 'wb')

10. **rb+ :**Opens the file for both reading and writing in binary format.

    Eg: file = open('filename.txt', 'rb+')

File close() method

After you finish working with a file, it's important to close it using the close() method. This ensures that any pending operations are completed and resources are freed up.

Syntax: file_handler.close()

Example:

```
file = open('myfile.txt', 'r')
# Perform operations on the file
file.close()
```

with statement to open and close files

The with statement in Python provides a convenient way to open and automatically close a file. It ensures that the file is properly closed, even if an exception occurs within the with block.

Syntax :

```
with open('filename.txt', 'mode') as file:
  # Perform operations on the file
  # ...
```

- with open('filename.txt', 'mode') as file: The with statement is used to open a file. Inside the with block, you specify the file name 'filename.txt' and the desired mode ('r', 'w', 'a', or 'x'). The open() function returns a file object that is assigned to the variable file.

Example:

```
# Writing to a file using 'with' statement
with open('myfile.txt', 'w') as file:
file.write('Hello, world!')
# Reading from a file using 'with' statement
with open('myfile.txt', 'r') as file:
    content = file.read()
    print(content)
```

Using the with statement eliminates the need to explicitly call the close() method. It ensures that the file is properly closed, improving code readability and reducing the chances of resource leaks.

File object attributes

When the Python open() function is called, it returns a file object called a file handler. Using this file handler, you can retrieve information about various file attributes.

Attribute	Description
file_handler.closed	It returns a Boolean True if the file is closed or False otherwise.
file_handler.mode	It returns the access mode with which the file was opened.
file_handler.name	It returns the name of the file.

Example,

```
file_handler = open("computer.txt", "w")
print(f"File Name is {file_handler.name}")
print(f"File State is {file_handler.closed}")
print(f"File Opening Mode is {file_handler.mode}")
```

Output:

File Name is computer.txt
File State is False
File Opening Mode is w

File methods to Read data from the file

When you use the open() function a file object is created.

The read methods are:

1. **read()**: This method reads the entire contents of the file as a string.

Syntax:file_handler. read([size])

If size is mentioned it will read the contents up to a size and return it as a string.

Example:

```
with open('myfile.txt', 'r') as file:
    content = file.read()
    print(content)
```

This will read the contents of the file 'myfile.txt' and assign it to the variable content. The print() statement will then display the content on the console.

2. **readline():** This method reads a single line from the file as a string. It reads up to and including the newline character ('\n').

Syntax: file_handler.readline()

Example:

```
with open('myfile.txt', 'r') as file:
   line = file.readline()
   print(line)
```

This will read the first line of the file 'myfile.txt' and assign it to the variable line. The print() statement will then display the line on the console.

3. **readlines():** This method reads all lines from the file and returns them as a list of strings. Each line is a separate item in the list.

Syntax: file_handler.readlines()

Example:

```
with open('myfile.txt', 'r') as file:
   lines = file.readlines()
   for line in lines:
      print(line)
```

This will read all lines from the file 'myfile.txt' and store them in the lines list. The for loop then iterates over each line and prints it on the console.

File methods to Write data to the file

They are:

1. **write():** This method writes a string to the file and returns the number of characters written.

Syntax: file_handler. write(string)

Example:

```
with open('myfile.txt', 'w') as file:
file.write('Hello, world!')
```

This will create a file named 'myfile.txt' and write the string 'Hello, world!' to it.

2. **writelines():** This method writes a list of strings to the file, where each string represents a line.

Syntax: file_handler.writelines(sequence)

Example:

```
with open('myfile.txt', 'w') as file:
   lines = ['Line 1', 'Line 2', 'Line 3']
file.writelines(lines)
```

This creates a file 'myfile.txt' and write the lines 'Line 1', 'Line 2', and 'Line 3' to it.

tell() method

Syntax: file_handler.tell()

Purpose: tell() is used to determine the current position within a file. It returns the current file position as an integer. The integer represents the number of bytes from the beginning of the file.

seek() method

Syntax: file_handler. seek(offset, from_what)

file_handler: file object

offset: It specifies the number of bytes to move the file pointer.

from_what (optional): It indicates the reference position from where the offset is applied. It can take the following values:

0 (default): The offset is relative to the beginning of the file.

1: The offset is relative to the current position.

2: The offset is relative to the end of the file.

Purpose:seek() method is used to move the file pointer to a specified position within the file allowing to read or write data from a specific location.

Python Program to Count the Occurrences of Each Word and Also Count the Number of Words in a text File.

```
occur=dict()
wcnt=0
withopen('file1.txt','r') asfp:
    forlineinfp:
        words=line.split()
        wcnt+=len(words)
        forwordinwords:
            occur[word] =occur.get(word,0)+1
    print (f"Count of all characters in the given string is :\n  {occur}")
```

File Content:

my country is India

I love my country

Output:

Count of all characters in the given string is :

{'my': 2, 'country': 2, 'is': 1, 'India': 1, 'I': 1, 'love': 1}

Object-Oriented Programming

The basic idea of Object-oriented programming (OOP) is that we use objects to model real-world things that we want to represent inside our programs and provide a simple way to access their functionality. An Object-oriented program is much easier to modify and maintain. So, Object-oriented programs require less work to maintain over time. Object-oriented programming results in code reuse, cleaner code, better architecture, abstraction layers, and fewer programming bugs. Python provides full support for Object-oriented programming, including encapsulation, inheritance, and polymorphism.

Classes and Objects:

A **Class** is like an object constructor, or a "blueprint" for creating objects. Classes provide a means of bundling data and functionality together.

Create a Class in Python :Syntax: Class Definition

```
class ClassName:
    # Statement
```

Classes are defined by using the class keyword, followed by the ClassName and a colon. The class binds the data members and methods into a single unit.

An **Object** is an instance of a Class. An object consists of :

- State: It is represented by the attributes of an object.
- Behaviour: It is represented by the methods of an object.
- Identity: It gives a unique name to an object and enables one object to interact with other objects.

Create Object of a Class :Syntax: Object Definition

```
obj = ClassName(arguments)
```

The object is created using the class name. When we create an object of the class, it is called instantiation.

A **constructor** is a special method used to create and initialize an object of a class. Using the def __init__(self): method we can implement constructor to initialize the object. The process is called Object initialization. The __init__() method never returns a value.

Syntax:

```
def __init__(self,arg1,arg2,…,argn):
        statements(s)
```

The self-parameter refers to the current instance of the class and accesses the class variables. self must be the first parameter of any function which belongs to the class. It does not have to be named self , you can call it whatever you like. Whenever we call a method using an object, the object itself is automatically passed in as the first parameter to the self parameter variable.

Accessing the members of class : Class attributes and methods are accessed using object and dot operator.

Syntax: object_name.attribute_name
Syntax: object_name.method_name()

```
#class and object
classMobile:
    def__init__(self):
        print("Inside the constructor")
    defrecieve_msg(self):
        print("Message recieved")
    defsend_msg(self):
        print("Message sent")
defmain():
    c1=Mobile()
    c1.recieve_msg()
    c1.send_msg()
if__name__=="__main__":
    main()
```

```
#class with parameter
classMobile:
    def__init__(self,name):
        self.name=name
        print(f"Inside the constructor {self.name}")
    defrecieve_msg(self):
        print(f"Messagerecieved from {self.name}")
    defsend_msg(self):
        print(f"Message sent from {self.name}")
defmain():
    c1=Mobile("Nokia")
    c1.recieve_msg()
    c1.send_msg()
if__name__=="__main__":
    main()
```

Classes with Multiple Objects

Multiple objects for a class can be created while attaching a unique copy of data attributes and methods of the class to each of these objects. In the below example, c1 and c2 are two objects of Mobile class.

```
#creating multiple objects
classMobile:
    def__init__(self,name):
        self.name=name
        print(f"Inside the constructor {self.name}")
    defrecieve_msg(self):
        print(f"Messagerecieved from {self.name}")
    defsend_msg(self):
        print(f"Message sent from {self.name}")
defmain():
    c1=Mobile("Nokia")
    c1.recieve_msg()
    c1.send_msg()
    c2=Mobile("Samsung")
    c2.recieve_msg()
    c2.send_msg()
if__name__=="__main__":
    main()
```

```
#Program to demonstrate the use of Default parameters in Methods
classMobile:
    def__init__(self,name="Nokia"):
        self.name=name
        print(f"Inside the constructor {self.name}")
    defrecieve_msg(self):
        print(f"Messagerecieved from {self.name}")
    defsend_msg(self):
        print(f"Message sent from {self.name}")
defmain():
    c1=Mobile("Samsung")
    c1.recieve_msg()
    c1.send_msg()
    c2=Mobile()
    c2.recieve_msg()
    c2.send_msg()
if__name__=="__main__":
    main()
```

In the mobile class, the paramenters of __init__() method have default values. If no arguments are mentioned in Mobile() while creating an instance, then the default value 'Nokia' is set for the 'name' attribute. If any values are specified in Mobile(), then the default value will be over-written with latest values.

Using Objects as Arguments

An object can be passed to a calling function as an argument. An object of a class as an argument, you have access to all the data attributes attached to that object.

Program to Demonstrate Passing of an Object as an Argument to a Function Call

```
classDog:
    def__init__(self,b,c):
        self.breed=b
        self.color=c

defdisplay(d):           # outside the class
        print(f"Dog breed is {d.breed}")
        print(f"Dogcolor is {d.color}")
d1=Dog("Bulldog", "Black")
display(d1)

The class Dog has breed and color attributes. Display() function
receives an object of Dog class d1 as parameter.
```

Returning Object from a method

To return an object from a method in Python, you can simply use the return statement followed by the object you want to return.

```
#using objects as argument
classDog:
    def__init__(self,b,c):
        self.breed=b
        self.color=c

    defdisplay(self):
        returnself
defmain():
    d1=Dog("Bulldog", "Black")
    obj=d1.display()
    print(f"Breed is {obj.breed}color is {obj.color}")
    print(id(d1))
    print(f"d1 is an instance of Dog class {isinstance(d1,Dog)}")
if__name__=="__main__":
    main()

Output:
Breed is Bulldog color is Black
2047374553296
d1 is an instance of Dog class True
```

The **id()** function is used to find the identity of the location of the object in memory. The syntax for id() function is, **id(object)**
This function returns the "identity" of an object. This is an integer (or long integer), which is guaranteed to be unique and constant for this object during its lifetime.

You can check whether an object is an instance of a given class or not by using the **isinstance()** function. The syntax for isinstance() function is, **isinstance (object, classinfo)** where the object is an object instance and classinfo can be a class, or a tuple containing classes, or other tuples. The isinstance() function returns a Boolean stating whether the object is an instance or subclass of another object.

Encapsulation

Encapsulation is the process of combining variables that store data and methods that work on those variables into a single unit called class. In Encapsulation, the variables are not accessed directly; it is accessed through the methods present in the class. Encapsulation ensures that the object's internal representation (its state and behavior) are hidden from the rest of the application. Thus, encapsulation makes the concept of data hiding possible.

Using Private Instance Variables and Methods

Private instance variables or private methods can be accessed within the same class and can't be seen outside. Mangling is a mechanism that allows class attributes to be transformed to avoid naming conflicts between parent and child classes. To apply name mangling to an attribute, you prefix it with two underscores (__) but do not end it with more underscores.

```
class mclass:
    __privateVariable1 = 2018;                  #private variable
    def __privateMethod1(self):                 #private method
        print("I'm inside the class mainclass which this is private method")
    def iclass(self):
        print("Private Variable: ",mclass.__privateVariable1)
        self.__privateMethod1()
foo = mclass()
foo.insideclass()
```

Inheritance

Inheritance allows us to define a class that inherits all the methods and properties from another class. **Parentclass** is the class being inherited from, also called base class. **Child class** is the class that inherits from another class, also called derived class.

Benefits of inheritance

- It provides Code reusability, readability, and scalability.
- It reduces code repetition. All the standard methods and attributes in the parent class are accessible by the child derived.
- By dividing the code into multiple classes, the applications look better, and error identification is easy.

Syntax:

```
Class BaseClass:
<Body>
Class DerivedClass(BaseClass):
<Body>
```

To create a derived class, you add a BaseClass after the DerivedClass within the parenthesis followed by a colon.

Classes without a base class name are implicitly derived directly from the class **object**. Leaving off the class object from the base class name is just shorthand for specifying that object is the base class.

Accessing inherited variables and methods

When the derived class object is constructed, the base class is also remembered. This is used for resolving variable and method attributes. If a requested attribute is not found in the derived class, the search proceeds to look in the base class. Inherited variables and methods are accessed just as if they had been created in the derived class itself.

Single Inheritance:

When a child class inherits from only one parent class, it is called single inheritance.

```
Class BaseClass:
<Body>
Class DerivedClass(BaseClass):
<Body>
```

Program to demonstrate Single Inheritance

```
classEmployee:
    x = 10
    deffunc_msg(self):
         print('Welcome to Employee')

classDepartment(Employee):
    deffunc_message(self):
        print('Welcome to Department')
            print('This  is  inherited  from
Employee')

emp = Employee()
print(emp.x)
emp.func_msg()

print('--------------')
dept = Department()
print(dept.x)
dept.func_message()
```

```
Output:
10
Welcome to Employee
--------------
10
Welcome to Department
This is inherited from Employee
```

emp = Employee() creates an instance of the Employee. Using this Object name, we can access the functions and properties of the Employee. dept = Department() – It creates an object of the Department. This object can access the members of Department class as well as its parent class.

Using super() Function and Overriding Base Class Methods / constructor overriding

In Python, constructor overriding refers to the ability to define a new constructor in a derived class that overrides the constructor of the base class. When a class inherits from another class, it can redefine the constructor to provide its own initialization logic or extend the initialization process defined in the base class. In a single inheritance, the built-in super() function can be used to refer to base classes. If you need to access the data attributes from the base class then you must explicitly call using super().

The syntax for using super() in derived class __init__() method definition looks like this:

super().__init__(base_class_parameter(s))

Its usage is shown below.

class DerivedClassName(BaseClassName):

def __init__(self, derived_class_parameter(s), base_class_parameter(s))

super().__init__(base_class_parameter(s))

```
classPerson:
  def__init__(self, fname, lname):
    self.firstname = fname
    self.lastname = lname

  defprintname(self):
    print(self.firstname, self.lastname)

classStudent(Person):
  def__init__(self, fname, lname):
    super().__init__(fname, lname)

x = Student("Mangalore", "University")
x.printname()
```

Output:

```
MangaloreUniversity
```

Method Overriding

Method Overriding in Python is an OOPs concept closely related to inheritance. When a child class method provides it's own implementation as the parent class method of the same name, parameters and return type, it is known as **method overriding.**

In this case, the child class's method is called the overriding method and the parent class's method is called the overriden method. An overriding method in a derived class may, in fact, want to extend rather than simply replace the base class method of the same name.

```
#Program to demonstrate Overriding
classPerson:
  def__init__(self, fname, lname):
    self.firstname = fname
    self.lastname = lname
  defprintname(self):
    print(self.firstname, self.lastname)

classStudent(Person):
  def__init__(self, fname, lname):
    super().__init__(fname, lname)
  defprintname(self):
    print("From Child class method calling parent class printname method")
    super().printname()

x = Student("Mangalore", "University")
x.printname()
```

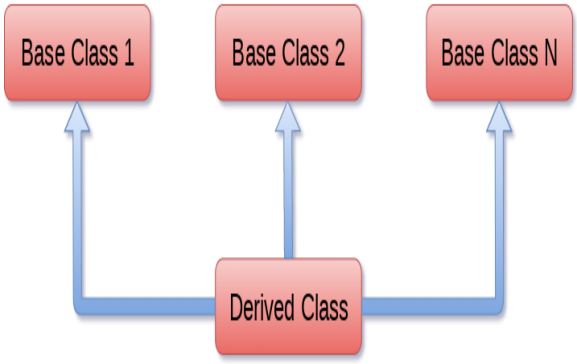
```
Output:
From Child class method calling parent class printname method
Mangalore University
```

printname() method is having same name and signature in both parent class and child class.
The super() function is useful for accessing the base class methods that have been overridden in a derived class without explicitly specifying the base class name.
The syntax for using the super() function to invoke the base class method is,
super().invoke_base_class_method(argument(s))

Multiple Inheritances

Multiple inheritance is a type of inheritance that allows a class can inherit from multiple base classes, incorporating their characteristics into a single derived class.

Syntax:




```
class Base1:
    <class-suite>
 class Base2:
    <class-suite>
.
.
.
class BaseN:
    <class-suite>
class Derived(Base1, Base2, ..... BaseN):
    <class-suite>
```

Example:

```
class Calculation1:
    def Summation(self,a,b):
        return a+b;
class Calculation2:
    def Multiplication(self,a,b):
        return a*b;
class Derived(Calculation1,Calculation2):
    def Divide(self,a,b):
        return a/b;
d = Derived()
print(d.Summation(10,20))
print(d.Multiplication(10,20))
print(d.Divide(10,20))
```

Output:

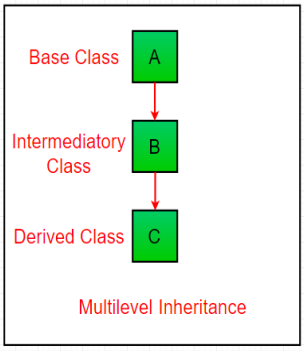
30

200

0.5

Multilevel Inheritance

In Python, we can also derive a class from the derived class. This form of inheritance is known as multilevel inheritance.



Here's the **syntax of the multilevel inheritance,**

```
class SuperClass:
    # Super class code here

class DerivedClass1(SuperClass):
    # Derived class 1 code here

class DerivedClass2(DerivedClass1):
    # Derived class 2 code here
```

Here, the DerivedClass1 class is derived from the SuperClass class, and the DerivedClass2 class is derived from the DerivedClass1 class.

```
#Program to demonstrate multilevel inheritance
class Manager:
        def final_review(self):
                print("Final Review")

class Reviewer(Manager):
        def review(self):
                print("Reviewing...")

class Writer(Reviewer):
        def writes(self):
                print("Writes the code")

obj = Writer()
obj.final_review()
obj.review()
obj.writes()
```

Output:

```
Final Review
Reviewing...
Writes the code
```

#Program to demonstrate super() function in multilevel inheritance

```
class Parent:
        def __init__(self):
                print('Parent - Hii')
        def age(self, a):
                print('Printing the age (Parent): ', a)

class Child(Parent):
        def __init__(self):
                print('Child - Hii')
                super().__init__()
        def age(self, a):
                print('Printing the age(Child): ', a)
                super().age(a +30)

class GrandChild(Child):
        def __init__(self):
                print('Grand Child - Hii')
                super().__init__()
        def age(self, a):
                print('Printing the age(Grand Child): ', a)
                super().age(a + 25)

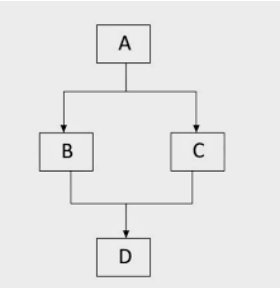
# Main function
if __name__ == '__main__':
        obj = GrandChild()
        obj.age(10)
```

Output

```
Grand Child - Hii
Child - Hii
Parent - Hii
Printing the age(Grand Child):  10
Printing the age(Child):  35
Printing the age (Parent):  65
```

Multipath Inheritance

When a class is derived from two or more classes which are derived from the same base class then such type of inheritance is called multipath inheritance.



Here class 'D' derived from classes 'B' and 'C', which are derived from same base class 'A'.

Here's the **syntax of the multipath inheritance,**

```
class ClassA:
    # Super class code here

class ClassB(ClassA):
    # Derived class B code here

class ClassC(ClassA):
    # Derived class C code here

class ClassD(ClassB, ClassC):
    # Derived class D code here
```

#Program to demonstrate Multipath Inheritance

```
class University:
    def __init__(self):
print("Constructor of the Base class")
    def display(self):
print(f"The University Class display method")

class Course(University):
    def __init__(self):
print("Constructor of the Child Class 1 of Class University")

super().__init__()
    def display(self):
print(f"The Course Class display method")
```

```
        super().display()

class Branch(University):
    def __init__(self):
print("Constructor of the Child Class
        super().__init__()
    def display(self):
print(f"The Branch Class display me
        super().display()

class Student(Course, Branch):
    def __init__(self):
print("Constructor of Child class of C
        super().__init__()
    def display(self):
print(f"The Student Class display me
        super().display()

# Object Instantiation:
ob = Student()  # Object named ob o
print()
ob.display()    # Calling the display m
```

Output:

```
Constructor of Child class of Course and Branch is called
Constructor of the Child Class 1 of Class University
Constructor of the Child Class 2 of Class University
Constructor of the Base class

The Student Class display method
The Course Class display method
The Branch Class display method
The University Class display method
```

Polymorphism

Poly means many and morphism means forms. Polymorphism means that you can have multiple classes where each class implements the same variables or methods in different ways.

Polymorphism takes advantage of inheritance in order to make this happen.

```
classAnimal:
    def__init__(self, name):
        self.name=name
    defmake_sound(self):
        pass
classDog(Animal):
    defmake_sound(self):
```

```
        return"Woof!"
classCat(Animal):
    defmake_sound(self):
        return"Meow!"
# Polymorphic function
defmake_animal_sound(animal):
    print(animal.make_sound())

# Creating different animal objects
dog=Dog("Buddy")
cat=Cat("Whiskers")

# Calling the polymorphic function with different animal objects
make_animal_sound(dog)   # Output: "Woof!"
make_animal_sound(cat)   # Output: "Meow!"
```

Operator Overloading

Operator overloading is a specific case of polymorphism, where an operator can have different meaning when used with operands of different types. By implementing specific magic methods in the class, we can define the behavior of operators when applied to instances of your class.

Magic methods are predefined methods with special names surrounded by double underscores. Magic methods are called automatically by the Python interpreter in response to specific events or operations, such as object creation, attribute access, comparison, arithmetic operations, and more.

Magic Methods for Different Operators and Functions

Binary Operators		
Operator	**Method**	**Description**
+	__add__(self, other)	Invoked for Addition Operations
-	__sub__(self, other)	Invoked for Subtraction Operations
*	__mul__(self, other)	Invoked for Multiplication Operations
/	__truediv__(self, other)	Invoked for Division Operations
//	__floordiv__(self, other)	Invoked for Floor Division Operations
%	__mod__(self, other)	Invoked for Modulus Operations
**	__pow__(self, other[, modulo])	Invoked for Power Operations
<<	__lshift__(self, other)	Invoked for Left-Shift Operations
>>	__rshift__(self, other)	Invoked for Right-Shift Operations
&	__and__(self, other)	Invoked for Binary AND Operations
^	__xor__(self, other)	Invoked for Binary Exclusive-OR Operations
|	__or__(self, other)	Invoked for Binary OR Operations

```
classAddition:
    def__init__(self, a):
        self.a=a
# operator overloading
    def__add__(self, obj):
        returnself.a+obj.a
ob1=Addition(1)
ob2=Addition(2)
ob3=Addition("python")
```

```
ob4=Addition('programming')

print(ob1+ob2)   #output : 3
print(ob3+ob4)   #output : pythonprogramming
```

Method Overloading:

Two or more methods have the same name but different numbers of parameters or different types of parameters, or both. These methods are called overloaded methods and this is called method overloading.

We can achieve method overloading in python by user defined function using “None” keyword as default parameter.

```
# code
def add(a=None, b=None):
        # Checks if both parameters are available
        # if statement will be executed if only one parameter is available
        if a != None and b == None:
                print(a)
        # else will be executed if both are available and returns addition of two
        else:
                print(a+b)
add(2, 3)
add(2)
```

The first parameter of “add” method is set to None. This will give us the option to call it with or without a parameter.

GUI in Python

In Python, there are several libraries available for creating Graphical User Interfaces (GUIs) to build interactive applications. Tkinter is one such library that allows to build GUI.

Tkinter: Tkinter is a standard Python library for creating GUI applications. It provides a set of widgets, such as buttons, labels, text boxes, and frames, which can be arranged to create a user interface. Tkinter is easy to use and has good documentation.

The root Window

In GUI programming, the root window is the main window or the top-level window in an application's window hierarchy. It serves as the parent or container for all other GUI elements, such as buttons, labels, menus, and other widgets.

Creating root window

Root window is created by creating an object of Tk class.

```
from tkinterimport  *
root=Tk()
root.mainloop()
```

Steps involved in creating a GUI application

To create a GUI application using Tkinter, the following steps are typically involved:

1. Import the Tkinter module: Start by importing the Tkinter module in your Python script to access the necessary classes and functions for GUI programming.
2. Create the root window: Create an instance of the Tk() class to create the main window or root window of your application.
3. Create a Canvas or Frame: Canvas and frames are child windows in root window. Canvas is used to draw shapes and Frame is used to place the widgets.
4. Add widgets: Use various Tkinter classes (such as Label, Button, Entry, etc.) to create and add widgets to the root window. These widgets represent the different GUI elements of your application.
5. Define event handlers: Specify the functions that should be executed when specific events occur, such as button clicks or key presses.
6. Start the event loop: Finally, call the mainloop() method on the root window to start the main event loop. This loop listens for events, handles user interactions, and keeps the GUI application running until the window is closed.

   Example:

```
fromtkinterimport*
root=Tk()
f=Frame(root,height=400,width=300)
defbutton_click():
    print("Button clicked!")
button=Button(f, text="Click Me!", command=button_click)
button.pack()
f.pack()
```

```
root.mainloop()
```

Working with Containers

A container is a component that is used as a place where drawings and widgets can be displayed.

The two main containers are:

Canvas: Canvas is a container is used to draw shapes like lines, arcs and circles

Frame: Frame is a container is used to display widgets like buttons, Labels, Entry etc.

Canvas

A canvas is used to create GUI to allow us to create and manipulate graphical objects, such as shapes, lines, images, and text.

```
fromtkinterimport*
root=Tk()
c=Canvas(root, width=400, height=300)
id=c.create_line(100,100,200,200)
c.pack()
root.mainloop()
```

Frame

Frame is a container is used to display widgets like buttons, Labels, Entry etc.

Creating a frame:

```
fromtkinterimport*
root=Tk()
f=Frame(root,height=400,width=300)
f.pack()
root.mainloop()
```

Layout Manager

Layout managers is used to position and arrange widgets within a GUI window. The three main layout managers available in Tkinter are:

1. Pack Layout Manager:

The pack() method is a simple layout manager that arranges widgets in a horizontal or vertical sequence. It automatically adjusts the size of widgets based on their content and places them one after another.

We can mention the position of the widget using ‘fill’ or ‘side’.

b.pack(fill=X)

b.pack(fill=Y)

‘fill’ takes values: X,Y,BOTH,NONE.

X represents that widget to occupy horizontally

Y represents that widget to occupy vertically

BOTH represents that widget to occupy in both direction.

NONE represents that widget to be displayed as it is.

'fill' option can take 'padx' and 'pady' options that represents the space between the widgets horizontally and vertically.

Example:

```
fromtkinterimport*
root=Tk()
f=Frame(root,height=300, width=300)
f.propagate(0)
f.pack()
b1=Button(f,text='Button 1',bg='blue')
b2=Button(f,text='Button 2',bg='red')
b1.pack(fill=X,padx=10,pady=10)
b2.pack(fill=X)
root.mainloop()
```

The pack() can take the option 'side', used to place the widgets side by side.
'side' can take the value LEFT,RIGHT,TOP,BOTTOM.
Default value is TOP.

```
fromtkinterimport*
root=Tk()
f=Frame(root,height=300, width=300)
f.propagate(0)
f.pack()
b1=Button(f,text='Button 1')
b2=Button(f,text='Button 2')
b1.pack(side=RIGHT)
b2.pack(side=TOP)
root.mainloop()
```

2. Grid Layout Manager:

The grid() method allows you to create a grid-like structure to position widgets in rows and columns. You can specify the row and column indices for each widget to determine its position within the grid. The grid() method provides flexibility in defining the layout and allows widgets to span multiple rows and columns.

```
fromtkinterimport*
root=Tk()
f=Frame(root,height=400,width=300)
f.pack()
label1=Label(f,text='Label 1')
label2=Label(f,text='Label 1')
label3=Label(f,text='Label 1')
label1.grid(row=0,column=0)
label2.grid(row=0,column=1)
label3.grid(row=1,column=0,columnspan=2)
root.mainloop()
```

3. Place Layout Manager:

The place() method allows you to specify the exact coordinates (x, y) of each widget within the window. With this layout manager, you have precise control over the positioning of widgets.

```
fromtkinterimport*
root=Tk()
f=Frame(root,height=400,width=300)
f.pack()
label1=Label(f,text='Label 1')
label2=Label(f,text='Label 2')
label3=Label(f,text='Label 3')
label1.place(x=10, y=10)
label2.place(x=50, y=50)
label3.place(x=100, y=100)
root.mainloop()
```

Widgets

Text

The Text widget is used to display and edit multiline text with different colors and fonts. It is possible to insert text into a Text widget, modify it and delete it. We can also display image in Text widget.

Creating a Text widget:

```
t=Text(root,width=20,height=10,font=(‘verdana’,14), fg=’vlue’)
```

t is the object of Text class. root is the object of root window.

Inserting text into Text widget using insert() method:

```
t.insert(END,'Text widget\n ')
```

END- represents that the given text is to be added at the end of the previous text.
CURRENT -represents that the given text is to be added at the current cursor position.

Display an image using image_create() method

```
img=PhotoImage(file='moon.gif')
t.image_create(END,image=img)
```

Marking some part of the text as a tag:

First specify the tag using tag_add() method and then apply color and font using config() method.

```
t.tag_add('start','2.5','2.11')
t.tag_config('start',background='red',foreground='white',font=('Verdana',2
0,'bold italic'))
```

start contains character from2nd row 5th character to 10th character

Adding a scrollbar to the Text widget:

We can add vertical or horizontal scrollbar. First create a scrollbar object and then configure it to the Text widget. Finally attach scroll bar to the root window.

```
s=Scrollbar(root,orient=VERTICAL,command=t.yview)
t.configure(yscrollcommand=s.set)
s.pack(side=RIGHT,fill=Y)
```

Example to demonstrate Text widget

```
fromtkinterimport*
root=Tk()
t=Text(root,width=20,height=10,font=('Verdana',14,'bold'),fg='blue')
t.insert(END,'Text widget\n ')
t.insert(CURRENT,'This text is \ninserted. ')
t.pack(side=LEFT)
img=PhotoImage(file='moon.gif')
t.image_create(END,image=img)
#create a tag with the name 'start'
t.tag_add('start','2.5','2.11')
#applycolors to the tag
t.tag_config('start',background='red',foreground='white',font=('Verdana',20
,'bold italic'))
#create a scrollbar
s=Scrollbar(root,orient=VERTICAL,command=t.yview)
#attach scroll bar to the Text widget
t.configure(yscrollcommand=s.set)
#attach scroll bar to the root window
s.pack(side=RIGHT,fill=Y)
root.mainloop()
```

Label

Label represents text that is displayed in the frame or container. Display text that cannot be modified.

To create a label in a Python GUI using Tkinter, we use the Label object.

```
#Label
fromtkinterimport*
root=Tk()
f=Frame(root,height=300, width=300)
f.propagate(0)
f.pack()
label=Label(f, text="Hello, World!")
label.pack()
root.mainloop()
```

Difference between Label and Text

- Label represents static text for viewing purpose. It is non-editable.

Creating a label: `label=Label(root, text="Hello, World!")`

- Text widget can have multiline text allowing insert, delete and modify actions on text.

Creating a Text: `t=Text(root,width=20,height=10)`

Button

A push button is a component that performs some action when clicked.

Creating a button:

```
b=Button(f,text='My Button', width=15, heigh=3, fg='blue')
```

b is the object of Button class. f represents that the button is shown on the frame.
'text' option represents the text to displayed on the Button.
Width and height of the button can be specified.
Foreground, background, activeforeground and activebackground attributes can be specified.

Handling click Event:

a. **Using bind()**

We link the mouse left button with buttonClick() method, using bind() method.

```
b.bind('<Button-1>',buttonClick)
```

<Button-1> represents the mouse left button. <Button-2> represents the mouse middle button. <Button-3> represents the mouse right button. <Enter> represents the event handler method when mouse pointer is placed on the Button.

```
#button
fromtkinterimport*
root=Tk()
f=Frame(root,height=300, width=300)
f.propagate(0)
f.pack()
defbuttonClick(event):
    print('Button is clicked')
b=Button(f,text='My Button', width=15, heigh=3, fg='blue')
b.bind('<Button-1>',buttonClick)
b.pack()
root.mainloop()
```

b. **Using command option:**

Command option in the Button constructor link the push button with event handler function.

```
#button
fromtkinterimport*
root=Tk()
f=Frame(root,height=300, width=300)
f.propagate(0)
f.pack()
defbuttonClick():
    print('Button is clicked')
b=Button(f,text='Submit',width=15,height=3,fg='blue',command=buttonClick)
b.pack()
root.mainloop()
```

Entry

The Entry widget in Tkinter is used to create a single-line input field where users can enter text or numeric values.

Creating an Entry widget: e1 = Entry(f, width=25,fg='blue',show='*')

e1 is the Entry widget object on the frame object f. Width is the width of the Entry widget and fg is the foreground color pf text. show=* is useful when user wants to hide text like password.

get() : Returns the entry's current text as a string.Eg: e1.get() returns the string.

Event handling: After typing the text in Entry widget, user presses the Enter button. This event can be linked using bind() method as, `e1.bind("<Return>",display)`

The event is passed to the display() method and method can contain the code to be executed during the event.

```
#Entry widget
fromtkinterimport*
root=Tk()
f=Frame(root,height=300, width=300)
f.propagate(0)
f.pack()
e1=Entry(f, width=25,fg='blue')
e1.place(x=50,y=100)
defdisplay(event):
    str=e1.get()
    print(str)
e1.bind("<Return>",display)
root.mainloop()
```

Listbox

List books is useful to display a list of items in a box so that the user can select one or more items.

Creating a

Listbox:`box1=Listbox(f,height=8,width=20,activestyle=UNDERLINE,selectmode=SINGLE)`

box1 list box object. **Height** and **width** of list box can be specified. The option **activestyle** indicates the appearance of the selected item which may take 'underline' 'dotbox' or 'none'; the default value is 'underline'.

The option **selectmode** may take any of the following values:

Browse: It is the default value of select mode. We can select one item out of the list box. If we click on the item and then drag to a different item, the selection will follow the mouse.

Single: This represents that we can select only one item from the available list of items.

Multiple: we can select one or more number of items at once by clicking on the items. If the item is already selected, clicking the second time on the item will unselect it.

Extended: We can select any adjacent group of items at once by clicking on the first item and dragging to the last item.

Inseting items into the list using insert() method:

```
box1.insert(0,'Mysore')
```

Binding ListboxSelect event with a method:

```
box1.bind('<<ListboxSelect>>',on_select)
```

When user selects any items in the listbox, the method on_select will be called.

```
#Listbox
fromtkinterimport*
root=Tk()
f=Frame(root,height=300, width=300)
f.propagate(0)
f.pack()
box1=Listbox(f,height=8,width=20,activestyle=UNDERLINE,selectmode=SINGLE)
box1.insert(0,'Mysore')
box1.insert(1,'Bangalore')
box1.insert(2,'Delhi')
box1.pack()

defon_select(event):
    lst=[]
    indexes=box1.curselection()
    foriinindexes:
        lst.append(box1.get(i))
    print(lst)

box1.bind('<<ListboxSelect>>',on_select)
root.mainloop()
```

Checkbutton

Checkbutton is used when user want to select one or more options from available group of options. Checkbuttons are displayed in the form of square shaped boxes.

Creating a Checkbutton:

```
c1=Checkbutton(f, text='Java',variable=var1,command=display)
```

c1 is a Checkbutton object, to be displayed on the frame. Height, width, fg, and bg can be specified. text represents the text to be displayed after the Check button. The option 'variable' represents an object of class IntVar() class. 'command' represents the method to be called when the user clicks the check button.

The class **IntVar** is used to know the state of the check button, whether clicked or not.
Object creation: var1=IntVar()
When the check button is selected, var1 will be 1, otherwise 0.
Retrieving the value from var1 is done using **get()** method as : x=var1.get()

```
fromtkinterimport*
root=Tk()
f=Frame(root,height=300, width=300)
f.propagate(0)
f.pack()
var1=IntVar()
defdisplay():
    ifvar1.get():
        print('Checkbutton is checked')
    else:
        print('Checkbutton is unchecked')
c1=Checkbutton(f, text='Java',variable=var1,command=display)
c1.pack()
root.mainloop()
```

Radiobutton

Radiobutton is used to select only one option from a group of options. A radiobutton is displayed as round shaped button.

Creating a radiobutton:

```
r1=Radiobutton(f, text='Male',variable=var1,command=display,value=1)
```

The 'text' represents the string to be displayed after the radio button.
'value' represents the value to be set when the radio button is clicked.
'variable' represents the object of IntVar class.: var=IntVar()
When user clicks the radio button, the value of 'var' will be set to the value in 'value' option. It is used to identify which radio button is clicked.

```
#Radiobutton
fromtkinterimport*
root=Tk()
f=Frame(root,height=300, width=300)
f.propagate(0)
f.pack()
var1=IntVar()
defdisplay():
    print('the value of the radio button is', var1.get())

r1=Radiobutton(f, text='Male',variable=var1,command=display,value=1)
r2=Radiobutton(f, text='Female',variable=var1,command=display,value=2)
r1.pack()
r2.pack()
root.mainloop()
```

Scrollbar

A scroll bar is used to scroll the text in other widget such as Text,Canvas, Frame or Listbox.

Types of scroll bars are: horizontal and vertical.

The horizontal scroll bar is used to view the text from left to right.

The vertical scroll bar is used to scroll the text from top to bottom.

Creating a Scrollbar:

Horizontal scrollbar:

```
h=Scrollbar(root,orient=HORIZONTAL,command=t.xview)
```

Vertical scrollbar:

```
v=Scrollbar(root,orient=VERTICAL,command=t.yview)
```

'h' represents Scrollbar object, which is to be placed on the root window.

'orient' represents the type of scrollbar.

It takes 2 values: HORIZONTAL or VERTICAL

'command' represents the method that is to be executed.

't.xview' represents that 'xview' is executed on the object 't'.

't' is an object of Text or Listbox etc.

Attaching the Scrollbar to the widget:

After creating the scrollbar it should be attached to the widget like Text or Listbox.

Attaching horizontal scrollbar:

```
t.configure(xscrollcommand=h.set)
```

'xscrollcommand' calls the set() method of horizontal scrollbar 'h'.

Attaching vertical scrollbar:

```
t.configure(yscrollcommand=v.set)
```

'xscrollcommand' calls the set() method of vertical scrollbar 'v'.

Attach scrollbar to the root windowis done using pack() or grid() method as:

```
h.pack(side=BOTTOM,fill=X)
v.pack(side=RIGHT,fill=Y)
```

```
#Adding Vertical Scrollbar to Text widget
fromtkinterimport*
root=Tk()
text=Text(root)
scrollbar=Scrollbar(root,command=text.yview)
text.configure(yscrollcommand=scrollbar.set)
text.pack(side=LEFT, fill=BOTH)
scrollbar.pack(side=RIGHT, fill=Y)
root.mainloop()
```

Spinbox

A Spinbox widget allows the user to select values from a given set of values. The values may be range of numbers or a fixed set of string.

Spinbox appears as a long rectangle attached with arrowheads pointing towards up and down. User can also edit the value being displayed in the Spinbox.

Creating a Spinbox:

```
s1=Spinbox(f, from_=1, to=10)
```

s1 is the Spinbox object, being placed in the frame.

'from_' represents the start value. 'to' represents the end value for the Spinbox.

After creating the spinbox place it in the root window using pack().

Get the current value using get() method as : value = s1.get()

```
fromtkinterimport*
root=Tk()
f=Frame(root,height=300, width=300)
f.propagate(0)
f.pack()
s1=Spinbox(f, from_=1, to=10)
s1.pack()
# Get the current value
value=s1.get()
print('selected value',value)
root.mainloop()
```